

CANDEX HANDBOOK

# API / Contrats

La référence de l'API REST Candex — conventions, enveloppe de réponse, codes de statut, en-têtes, authentification, et les endpoints par ressource.

Rédigé par
**Eliott Barbet**

Version 0.23.0
novembre 2026
candex.app

# API / Contrats

Voici la référence de contrat de `candex-api`. Elle s'appuie sur Back-end (comment l'API est construite) et liste les endpoints par ressource. Le **contrat vivant et exhaustif** — schémas requête/réponse, exemples et essai en direct — est **Swagger UI** sur `/swagger-ui.html` ; cette page en est la carte stable.

## Conventions

- **Enveloppe** — chaque réponse est un `Result<T>` côté Java, sérialisé en charge utile brute en cas de succès ou en `Error` brute en cas d'échec (voir Back-end).
- **Codes de statut** — issus d'exceptions typées (`404` / `400` / `409` / `422`…) ; voir Gestion d'erreurs du Back-end.
- **En-têtes** — les réponses de liste portent `X-Total-Count` ; les créations renvoient `201` avec un en-tête `Location`.
- **Authentification** — envoyez un token d'identité Firebase via `Authorization: Bearer <token>`.
- **Accès** — par endpoint, dans les tableaux : `USER_OR_ADMIN` = tout utilisateur authentifié, `ADMIN` = admin seulement, `PUBLIC` = sans auth, **own** = l'utilisateur authentifié n'agit que sur ses propres enregistrements.
- **Chemin de base** — les ressources sont servies sous la base de l'API (par exemple `/api/drinks`).

## Appeler l'API

Une lecture envoie l'ID token Firebase et reçoit le DTO brut :

```
curl https://<api-host>/api/users/firebase/4q8ESFox1Scd8WMT438M5UlZGMy1 \
  -H "Authorization: Bearer $FIREBASE_ID_TOKEN"
```

```
{
  "id": 42,
  "firebaseUid": "4q8ESFox1Scd8WMT438M5UlZGMy1",
  "username": "therealinvestigator",
  "displayedName": "Investigator",
  "email": "investigator@candex.app",
  "countryCode": "FR",
  "emailVerified": true,
  "joinedInVersion": "1.1.10"
}
```

L'inscription (`POST /users`) est ce que l'app envoie juste après avoir créé le compte Firebase. `id`, `emailVerified` et `joinedInVersion` sont fixés par l'API, jamais par l'appelant :

```
{
  "firebaseUid": "4q8ESFox1Scd8WMT438M5UlZGMy1",
  "username": "therealinvestigator",
  "displayedName": "Investigator",
  "email": "investigator@candex.app",
  "countryCode": "FR",
  "birthDate": "2001-07-15"
}
```

Elle répond `201 Created`, un en-tête `Location: /users/42` et l'utilisateur créé. Débloquer une boisson est l'écriture la plus simple de l'API :

```
{
  "userId": 42,
  "drinkId": 7
}
```

Elle répond avec le déblocage enregistré et son horodatage serveur :

```
{
  "userId": 42,
  "drinkId": 7,
  "unlockedAt": "2026-11-12T20:52:52"
}
```

Tous les échecs ont la même forme, quel que soit l'endpoint — ici, l'inscription avec un nom d'utilisateur déjà pris :

```
{
  "name": "UniqueConstraintViolationException",
  "message": "A UserEntity with username = 'therealinvestigator' already exists.",
  "status": 409,
  "timestamp": "2026-11-12 20:52:52"
}
```

Le client peut se baser sur `status` pour la catégorie d'erreur et sur `name` pour le cas exact.

# Catalogue

## Drinks — `/drinks`

| Méthode | Chemin | Accès | But |
|---|---|---|---|
| GET | /drinks | USER_OR_ADMIN | Lister les boissons |
| GET | /drinks/{id} | USER_OR_ADMIN | Une boisson |
| POST | /drinks | ADMIN | Créer une boisson |
| POST | /drinks/import | ADMIN | Import en masse |
| POST | /drinks/import/diff | ADMIN | Prévisualiser un import face au catalogue actuel |
| POST | /drinks/import/sync | ADMIN | Appliquer un import |
| PUT | /drinks/{id}/shift | ADMIN | Déplacer la position World Dex |
| PUT | /drinks/{id}/swap | ADMIN | Échanger deux positions World Dex |
| DELETE | /drinks/{id} | ADMIN | Supprimer une boisson |

## Cans — /cans

| Méthode | Chemin | Accès |
|---|---|---|
| GET | /cans • /cans/{id} | USER_OR_ADMIN |
| POST | /cans | ADMIN |
| DELETE | /cans/{id} | ADMIN |

## Données de référence (lectures)

Les ressources de référence du catalogue exposent des lectures liste et par-id : **Brands** /brands, **Flavors** /flavors et **Flavor categories** /flavor-categories, **Colors** /colors (+ /colors/extract), **Drink categories** /drink-categories, **Can formats** /can-formats, et **App versions** /app-versions (+ /app-versions/current).

# World Dex — `/world-dex`

| Méthode | Chemin | Accès | But |
|---|---|---|---|
| GET | `/world-dex/drinks/{id}/history` | USER_OR_ADMIN | Historique de position d'une boisson |
| GET | `/world-dex/drinks/{id}/stats` | USER_OR_ADMIN | Stats Dex d'une boisson |
| GET | `/world-dex/snapshot` | USER_OR_ADMIN | Instantané du Dex |
| GET | `/world-dex/movers` | USER_OR_ADMIN | Mouvements récents |
| GET | `/world-dex/positions/{position}/occupants` | USER_OR_ADMIN | Qui occupe une position |

# Utilisateurs & collection

## Users — `/users`

| Méthode | Chemin | Accès | But |
|---|---|---|---|
| GET | /users | ADMIN | Lister les utilisateurs |
| GET | /users/{id} | own | Un utilisateur |
| GET | /users/firebase/{firebaseUid} | own | Un utilisateur par UID Firebase |
| GET | /users/{id}/drinks | own | Les boissons débloquées d'un utilisateur |
| GET | /users/exists | PUBLIC | Vérifier l'existence |
| POST | /users | PUBLIC | S'inscrire |
| PUT • PATCH | /users/{id} | own | Mettre à jour un utilisateur |
| DELETE | /users/{id} | own | Supprimer un utilisateur |

### Unlocked drinks — `/user-unlocked-drinks`

| Méthode | Chemin | Accès | But |
|---|---|---|---|
| GET | /user-unlocked-drinks/{userId} | own | Les déblocages d'un utilisateur |
| POST | /user-unlocked-drinks | own | Débloquer une boisson |
| DELETE | /user-unlocked-drinks/{userId}/{drinkId} | own | Retirer un déblocage |

## Admin & opérations

- **Console SQL** — `POST /admin/...` (ADMIN) : exécuter une requête en lecture seule via le rôle dédié (voir Base de données & Modèle).
- **Reports** — `POST /reports` : générer un rapport.

# Le flux d’import en masse

Importer un lot de boissons est un pipeline en trois étapes, réservé aux admins :

1. `POST /drinks/import` — soumettre le payload.
2. `POST /drinks/import/diff` — prévisualiser les changements face au catalogue actuel.
3. `POST /drinks/import/sync` — les appliquer.

# Où aller ensuite

- Comment l’API est construite → Back-end
- Schémas vivants & essai en direct → **Swagger UI** (`/swagger-ui.html`)